Lab 02 — Troubleshooting Kubernetes Like an SRE

ID: K8S-LAB-02 · **Category:** Troubleshooting · **Level:** Engineer · **Duration:** 75 min

Focus: kubectl get, describe, logs, events, journalctl, investigation methodology

Objective

Master the canonical SRE investigation workflow: get → describe → logs → events → runtime — and apply it to six recurring failure modes.

Concepts Covered

• Why describe almost always beats logs as a first step

• How Events are TTL'd and where they live

• Difference between application logs (stdout) and kubelet logs (journald)

• Cluster-wide vs namespaced views

Prerequisites

• A working cluster from Lab 1

• kubectl context set to that cluster

Architecture

Each step narrows the search space. Don't skip describe — it folds events, status conditions and last-termination state into one screen.

```
kubectl get   ->  what state is the object in?
     |
     v
kubectl describe -> events + conditions + last-state
     |
     v
kubectl logs (-p) -> what did the container say (or its previous incarnation)?
     |
     v
kubectl get events -> what did the cluster say around the same time?
     |
     v
journalctl -u kubelet / crictl ps -> node-level reality check
```

Lab Steps

01. Create a deliberately broken Pod

Reproduces an ImagePullBackOff in a controlled way so you can practice the workflow.

```
kubectl run badimg --image=nginx:doesnotexist
kubectl get pod badimg
```

Expected: STATUS=ImagePullBackOff or ErrImagePull within ~30s.

Internals: kubelet asks containerd to pull the image. containerd retries with exponential backoff and reports failure back through the CRI status.

02. Step 1 — describe

The Events section at the bottom shows the registry error verbatim.

```
kubectl describe pod badimg | tail -20
```

Expected: `Failed to pull image "nginx:doesnotexist": ... manifest unknown`

Internals: Events are stored as Event objects in the same namespace with a default TTL of 1h.

03. Step 2 — fix and verify

Pods are largely immutable, but image is one of the few fields you can patch in place.

```
kubectl set image pod/badimg badimg=nginx:1.27
kubectl get pod badimg -w
```

Expected: Status goes ContainerCreating → Running.

Internals: kubelet sees the spec diff, re-asks containerd to pull, restarts the container.

04. Reproduce CrashLoopBackOff

Forces a guaranteed crash so you can inspect previous-container logs.

```
kubectl run crasher --image=busybox --restart=Never -- /bin/sh -c 'exit 1'
```

Expected: RESTARTS climbs, STATUS becomes CrashLoopBackOff.

Internals: kubelet applies an exponential backoff (10s, 20s, 40s …) up to 5 minutes between restart attempts.

05. Step 3 — logs of the previous container

Without -p you'll get the current (not-yet-restarted) container; with -p you get the one that just died.

```
kubectl logs crasher -p
```

Expected: (empty in this trivial case — but in real apps you'd see the stack trace)

Internals: Logs are pulled from /var/log/containers/* via the CRI log endpoint.

06. Step 4 — cluster events

Gives you a cross-object timeline; invaluable when something cascades.

```
kubectl get events --sort-by=.lastTimestamp | tail -15
```

Expected: BackOff events for the crasher Pod, plus any unrelated cluster events.

Internals: kubectl get events is just `kubectl get -o wide events.v1.events.k8s.io`.

07. Step 5 — node-level reality check

When the API tells you everything is fine but pods still misbehave, drop down to the kubelet and the runtime.

```
sudo journalctl -u kubelet --since '5 min ago' | tail -30
sudo crictl ps
```

Expected: kubelet log entries about backing off, crictl shows the container states.

Internals: journald is the only place you'll see why kubelet itself failed (e.g. invalid kubeconfig, cgroup driver mismatch).

Common Errors

kubectl logs returns nothing — Cause: Container restarted; you need the previous instance · Fix: Add `-p` / `--previous`

describe shows no Events — Cause: Events older than 1h; TTL'd out · Fix: Increase --event-ttl on the API server or use kube-state-metrics + persistent store

`crictl: command not found` — Cause: Not installed on the node · Fix: `sudo apt install cri-tools` or use `nerdctl` against containerd directly

Troubleshooting

• Always start with describe; logs without context is guessing

• If the API server is slow, hit `--raw=/healthz` to confirm it's the API and not your kubectl

• When something is namespace-scoped, never forget `-n`

Enterprise Relevance

Senior SREs are not the people who know the most YAML — they're the ones who follow the same investigation loop every single time under pressure.

Validation

• You can recover a broken Pod without restarting the cluster

• You can tell whether a failure is application, scheduler, runtime or network in <5 min

Related Incidents & Guide Sections

Incidents: K8S-INC-3001, K8S-INC-3003

• Troubleshooting methodology

• CrashLoopBackOff

• ImagePullBackOff

Questions

Beginner

Q. What's the first command you run when a Pod is unhealthy?

A. kubectl describe pod <name> — events + conditions in one place.

Intermediate

Q. Why does `kubectl logs` return empty for a crashlooping pod?

A. Because the current container hasn't produced output yet; use `-p` for the previous instance.

Advanced

Q. How would you correlate a Pod's failure with what the kubelet was doing?

A. Take the Pod event timestamp, then `journalctl -u kubelet --since '<ts-30s>' --until '<ts+30s>'`.

Scenario

Q. A pod shows Running 1/1 but the app is returning 503s to clients. Where do you look?

A. Service endpoints (selector mismatch), readiness probe (passing but app broken), then app logs.

Viva

Q. Walk through your five-step kubectl investigation loop.

A. get → describe → logs (-p) → get events → kubelet/crictl on the node.